

Name: Gaurika Mittal

Batch: 15

Sap ID: 590034892

Experiment-17

Question

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, with **void return type** and with a **simple function call (without call by value/reference)**. Each function should read the required input, process it, and display the result.

1. Check whether it is a **Strong number**.
2. Check whether it is an **Armstrong number**.
3. Check whether it is a **Harshad number**.
4. Check whether it is a **Duck number**.

Algorithm

Main Program Algorithm

1. Start.
2. Display the menu:
 - 1. Check Strong Number
 - 2. Check Armstrong Number
 - 3. Check Harshad Number
 - 4. Check Duck Number
3. Ask the user to enter a choice.
4. Read the choice.
5. Use **switch-case** to perform the selected operation.
6. If choice is 1, call `strong()`.
7. If choice is 2, call `armstrong()`.
8. If choice is 3, call `harshad()`.
9. If choice is 4, call `duck()`.
10. For any other choice, display "Invalid choice".

11. Stop.

Algorithm for Strong Number

1. Start.
2. Input a positive integer n .
3. Store n in $temp$.
4. Initialize $sum = 0$.
5. Extract the last digit using $digit = temp \% 10$.
6. Find the factorial of the digit.
7. Add the factorial to sum .
8. Remove the last digit using $temp = temp / 10$.
9. Repeat steps 5–8 until $temp$ becomes 0.
10. If $sum == n$, display "Strong Number".
11. Otherwise, display "Not a Strong Number".
12. Stop.

Example: 145

$$1! + 4! + 5! = 1 + 24 + 120 = 145$$

Therefore, 145 is a Strong number.

Algorithm for Armstrong Number

1. Start.
2. Input a positive integer n .
3. Count the number of digits in n .
4. Store n in $temp$.
5. Initialize $sum = 0$.
6. Extract the last digit using $digit = temp \% 10$.
7. Calculate the digit raised to the power of the number of digits.
8. Add it to sum .
9. Remove the last digit using $temp = temp / 10$.
10. Repeat until $temp$ becomes 0.
11. If $sum == n$, display "Armstrong Number".
12. Otherwise, display "Not an Armstrong Number".
13. Stop.

Example: 153

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

Therefore, 153 is an Armstrong number.

Algorithm for Harshad Number

1. Start.
2. Input a positive integer n .
3. Store n in $temp$.
4. Initialize $sum = 0$.
5. Extract the last digit using $digit = temp \% 10$.

6. Add the digit to `sum`.
7. Remove the last digit using `temp = temp / 10`.
8. Repeat until `temp` becomes 0.
9. Check whether `n % sum == 0`.
10. If yes, display "Harshad Number".
11. Otherwise, display "Not a Harshad Number".
12. Stop.

Example: 18

Sum of digits = $1 + 8 = 9$

18 is divisible by 9.

Therefore, **18 is a Harshad number.**

Algorithm for Duck Number

1. Start.
2. Input a positive integer `n`.
3. Initialize `flag = 0`.
4. Extract each digit of the number.
5. If any digit is 0, set `flag = 1`.
6. Continue until all digits have been checked.
7. If `flag == 1`, display "Duck Number".
8. Otherwise, display "Not a Duck Number".
9. Stop.

Example: 1205 contains 0, so **1205 is a Duck number.**

Test Case Table

Test Case	Choice	Inputs	Expected Outputs	Actual Outputs	Results
1	1	145	Strong Number	Strong Number	Pass
2	1	123	Not Strong Number	Not Strong Number	Pass
3	2	153	Armstrong Number	Armstrong Number	Pass
4	2	123	Not Armstrong Number	Not Armstrong Number	Pass
5	3	18	Harshad Number	Harshad Number	Pass
6	3	19	Not Harshad Number	Not Harshad Number	Pass

7	4	1205	Duck Number	Duck Number	Pass
8	4	1235	Not Duck Number	Not Duck Number	Pass

C Program

```
#include <stdio.h>
```

```
void strong();
```

```
void armstrong();
```

```
void harshad();
```

```
void duck();
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    printf("1. Check Strong Number\n");
```

```
    printf("2. Check Armstrong Number\n");
```

```
    printf("3. Check Harshad Number\n");
```

```
    printf("4. Check Duck Number\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch(choice)
```

```
{
```

```
    case 1:
```

```
        strong();
```

```

        break;

    case 2:

        armstrong();

        break;

    case 3:

        harshad();

        break;

    case 4:

        duck();

        break;

    default:

        printf("Invalid choice\n");

}

return 0;

}

void strong()

{

    int n, temp, digit, sum = 0;

    int fact, i;

    printf("Enter a positive integer: ");

    scanf("%d", &n);

    temp = n;

    while(temp > 0)

    {

        digit = temp % 10;

```

```

    fact = 1;

    for(i = 1; i <= digit; i++)
    {
        fact = fact * i;
    }

    sum = sum + fact;

    temp = temp / 10;
}

if(sum == n)

    printf("%d is a Strong Number\n", n);

else

    printf("%d is Not a Strong Number\n", n);
}

void armstrong()
{
    int n, temp, digit, count = 0;

    int sum = 0, power, i;

    printf("Enter a positive integer: ");

    scanf("%d", &n);

    temp = n;

    while(temp > 0)
    {
        count++;

        temp = temp / 10;
    }

```

```

temp = n;
while(temp > 0)
{
    digit = temp % 10;
    power = 1;
    for(i = 1; i <= count; i++)
    {
        power = power * digit;
    }
    sum = sum + power;
    temp = temp / 10;
}
if(sum == n)
    printf("%d is an Armstrong Number\n", n);
else
    printf("%d is Not an Armstrong Number\n", n);
}

void harshad()
{
    int n, temp, digit, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {

```

```

        digit = temp % 10;

        sum = sum + digit;

        temp = temp / 10;
    }

    if(n % sum == 0)

        printf("%d is a Harshad Number\n", n);

    else

        printf("%d is Not a Harshad Number\n", n);
}

void duck()
{
    int n, temp, digit, flag = 0;

    printf("Enter a positive integer: ");

    scanf("%d", &n);

    temp = n;

    while(temp > 0)
    {
        digit = temp % 10;

        if(digit == 0)
        {
            flag = 1;

            break;
        }

        temp = temp / 10;
    }
}

```



```

if(flag == 1)

    printf("%d is a Duck Number\n", n);

else

    printf("%d is Not a Duck Number\n", n);

}

```

Compilation and Execution

```

(kali㉿kali)-[~]
└─$ gcc hdoc18.c -o hdoc18

(kali㉿kali)-[~]
└─$ ./hdoc18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong Number

(kali㉿kali)-[~]
└─$ gcc hdoc18.c -o hdoc18

(kali㉿kali)-[~]
└─$ ./hdoc18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 123
123 is Not a Strong Number

(kali㉿kali)-[~]
└─$ gcc hdoc18.c -o hdoc18

(kali㉿kali)-[~]
└─$ ./hdoc18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong Number

(kali㉿kali)-[~]
└─$ gcc hdoc18.c -o hdoc18

(kali㉿kali)-[~]
└─$ ./hdoc18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 123
123 is Not an Armstrong Number

```

```
(kali㉿kali)-[~]  
$ gcc hdoc18.c -o hdoc18  
  
(kali㉿kali)-[~]  
$ ./hdoc18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 3  
Enter a positive integer: 18  
18 is a Harshad Number
```

```
(kali㉿kali)-[~]  
$ gcc hdoc18.c -o hdoc18  
  
(kali㉿kali)-[~]  
$ ./hdoc18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 3  
Enter a positive integer: 19  
19 is Not a Harshad Number
```

```
(kali㉿kali)-[~]  
$ gcc hdoc18.c -o hdoc18  
  
(kali㉿kali)-[~]  
$ ./hdoc18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 4  
Enter a positive integer: 1205  
1205 is a Duck Number
```

```
(kali㉿kali)-[~]  
$ gcc hdoc18.c -o hdoc18  
  
(kali㉿kali)-[~]  
$ ./hdoc18  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 4  
Enter a positive integer: 1235  
1235 is Not a Duck Number
```